Software Architecture and Design Specification

Railway Reservation System

Software Engineering Laboratory Project, PES University

Team 11, Class 5A CSE

Version	Date	Status
1.2	10 November 2026	Submitted for review

Team

	Abhigyan Dutta	Akash Singh	Aditya Alur	Aditya G. Mule
SRN	PES2UG24CS019	PES2UG24CS042	PES2UG24CS052	PES2UG24CS031

Revision history

Ver	Date	Author	Change summary
0.1	05 Sep 2026	Abhigyan Dutta	Outline, scope and first cut of the functional requirements
0.2	06 Sep 2026	Aditya Alur	Booking, waitlist and refund requirements rewritten after the team walked through a Tatkal booking by hand
0.3	06 Sep 2026	Aditya G. Mule	Use case diagrams and the UC-03 flow added as Section 7
0.4	07 Sep 2026	Akash Singh	External interfaces filled in, load figures in Section 5 corrected
0.5	08 Sep 2026	Abhigyan Dutta	Requirement set cut back to what the team can build this semester, duplicates removed and IDs renumbered
1.0	09 Sep 2026	Abhigyan Dutta	SRS traceability matrix completed and requirement IDs frozen
1.0	13 Sep 2026	Abhigyan Dutta & Akash Singh	SAD v1.0: architecture, design, STRIDE threat model, API contracts, traceability to SRS completed
1.1	09 Nov 2026	Abhigyan Dutta	Stale placeholder note replaced, handheld open issue updated for the PWA, tooling updated to GitHub Projects and CI coverage; implementation deviations recorded in docs/DEVIATIONS.md
1.2	10 Nov 2026	Abhigyan Dutta	Sprint 2 complete: payment, tickets, refunds, cancellation, chart, handheld, administration and security built to this design; differences recorded in docs/DEVIATIONS.md (D14 onwards)

Approvals

Role	Name	Signature / Date
Course Coordinator	Prof. Pavithra	
Class Coordinator	Prof. Deena Hajim	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Railway Reservation System. It translates the requirements in the SRS (version 1.0, 09 September 2026) into component structure, interface contracts, data flow and technology choices. It is written for Team 11 developers and testers, and for faculty evaluation.

1.2 Scope

Covers all SRS in-scope functionality: passenger accounts, search, reservation with RAC/waitlist/Tatkal, fare and payment, cancellation with promotion, chart preparation, offline ticket verification, and administration. Three front ends (passenger web, counter, handheld) over one shared inventory. Out-of-scope items from SRS Section 1.2 remain out of scope.

1.3 Audience

Developers and testers on the team, the faculty evaluators, and anyone who will maintain or extend this system after the current semester.

1.4 Definitions

Term	Meaning
ADR	Architecture Decision Record, a short document capturing one design choice and its reasoning
STRIDE	Threat classification: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
DDD	Domain Driven Design, structuring code around business concepts rather than technical layers

Other terms (PNR, RAC, RBAC, TLS, WCAG, Tatkal) are defined in SRS Section 1.4.

2. Document Overview

2.1 How to use this document

Section 3 covers architecture: goals, component breakdown, chosen pattern, technology stack, risks, traceability to SRS, and security architecture including a STRIDE threat model. Section 4 covers design: module structure, sequence diagrams for two core flows, API contracts for two components, error handling, UX decisions, and open issues. Section 5 contains the appendices.

The component diagram is in Section 3.3 and the two sequence diagrams are in Section 4.2. Their editable sources (PlantUML and draw.io) and image files are in the project repository under diagrams. Differences between this design and the Sprint 1 and Sprint 2 implementation are recorded, with reasons, in docs/DEVIATIONS.md in the repository.

2.2 Related documents

Document	Relationship
SRS, Railway Reservation System, Team 11, v1.0	This SAD implements those requirements. IDs (RRS-F-xxx, RRS-NF-xxx, RRS-SR-xxx) are referenced throughout.
Software Test Plan (STP)	Will reference modules and interfaces defined here.
RTM (SRS Section 8)	Design reference column (DS-xxx) points into this document.

3. Architecture

3.1 Goals and constraints

Inventory integrity. No berth allocated twice (RRS-F-005). This is non-negotiable. Every other goal is secondary.

Financial correctness. One payment produces one ticket. Payment without ticket produces refund (RRS-F-012 to RRS-F-014). Payment adapter is idempotent.

Burst tolerance. Tatkal window spike: 50 req/s at p95 under 3 seconds (RRS-NF-001).

Offline capability. Ticket examiner works from downloaded chart with no network (RRS-F-018).

Data protection. Card data never stored (RRS-NF-005, RRS-SR-002). PNR lookups masked (RRS-SR-004).

Constraints: Strict consistency on inventory (row-level locking, not eventual consistency). All policy values (fares, quotas, slabs, Tatkal time) in configuration tables, not code. Payment via gateway hosted page only.

3.2 Stakeholders and concerns

Stakeholder	Primary concern
Passenger	Fast booking, honest availability, prompt ticket, refund on cancellation
Booking Clerk	Keyboard-driven speed, cannot issue unsupported ticket
Reservation Administrator	Change fare/quota rules with effective date, full audit trail
Ticket Examiner	Verify tickets offline from chart on device
Faculty evaluator	Separation of concerns, traceability, testability
Development team	Modularity so four people work in parallel

3.3 Component diagram

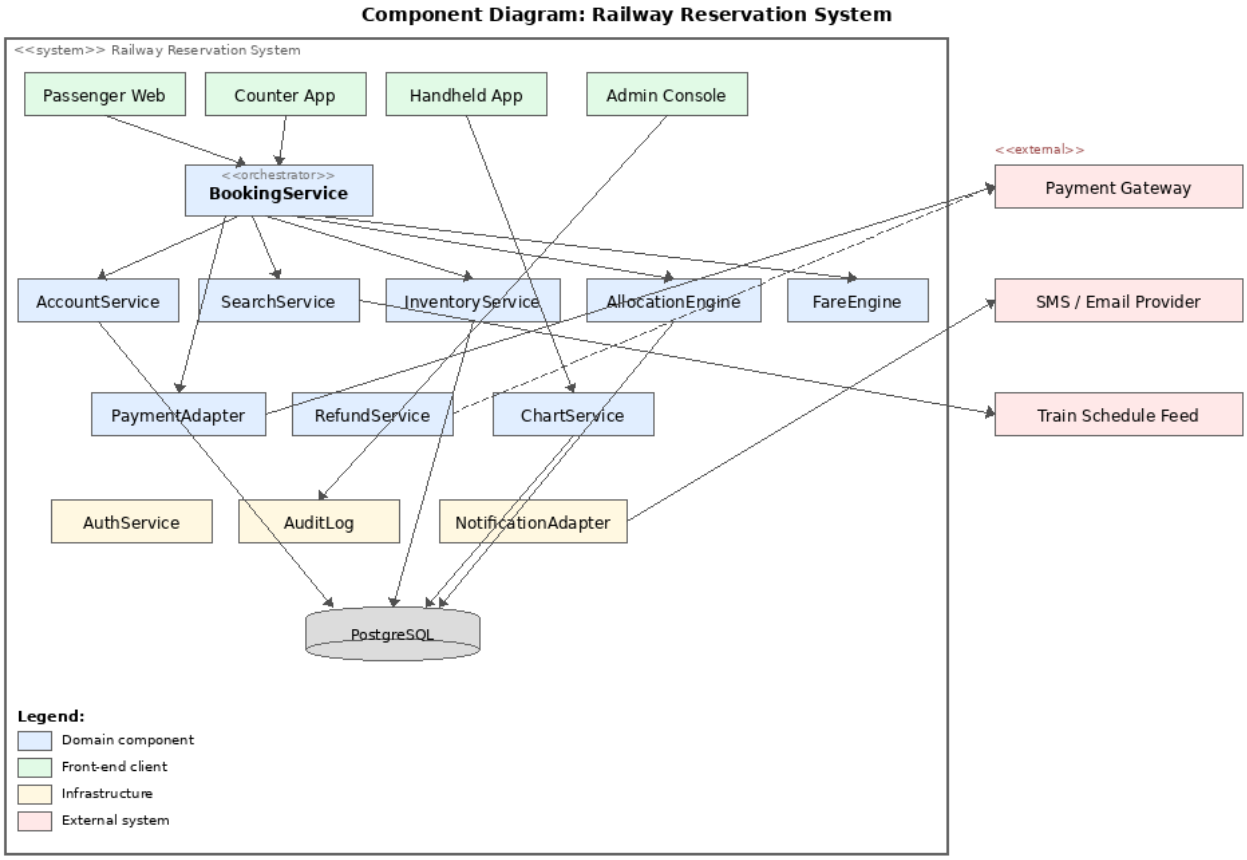


Figure 1. *UML component diagram showing domain modules, front-end clients, infrastructure, external systems and the database.*

Components (nine domain):

AccountService, *SearchService*, *InventoryService*, *AllocationEngine*, *FareEngine*, *BookingService*, *PaymentAdapter*, *RefundService*, *ChartService*.

Infrastructure:

AuthService, *AuditLog*, *NotificationAdapter*. *Front ends: Passenger Web*, *Counter App*, *Admin Console*, *Handheld App*.

External:

Payment Gateway, *SMS/Email Provider*, *Train Schedule Feed*.

3.4 Component descriptions

Component	Responsibility	SRS reqs
AccountService	Registration, OTP verification, profile storage	F-001, F-002
SearchService	Train and availability lookup from live inventory (max 30 s stale)	F-003, F-004
InventoryService	Owns seat inventory table. Holds berths, releases expired holds, confirms allocation. Enforces no-double-allocation via row-level locking.	F-005, F-006, NF-003
AllocationEngine	Confirmed / RAC / waitlist decision per quota. Tatkal window check. Waitlist promotion on cancellation.	F-007 to F-009, F-016
FareEngine	Computes fare breakup from configuration tables (base fare,	F-010

Component	Responsibility	SRS reqs
	reservation charge, taxes, convenience fee).	
BookingService	Orchestrates booking flow: hold, price, pay, confirm, generate PNR, notify.	F-012
PaymentAdapter	Gateway hosted page handoff. Receives and verifies signed callbacks. Idempotent.	F-011, F-013
RefundService	Slab-based refund on cancellation. Auto refund when payment succeeds but allocation fails.	F-014, F-015
ChartService	Prepares chart at configured time. Freezes allocation, publishes berths, cancels remaining waitlist. Serves chart download.	F-017, F-018
AuthService	RBAC, second factor for clerk/admin, session management, lockout after 5 failures.	SR-003, SR-005
AuditLog	Append-only log of privileged actions. Cannot be edited or deleted.	F-020
AdminConsole	Schedule, quota, fare rule maintenance. Impact preview before applying changes.	F-019
HandheldApp	PWA for the course demo: service worker caches the chart in IndexedDB, html5-qrcode library for QR scanning. Runs on Android Chrome offline. Production note: would wrap in Capacitor to produce a native APK with guaranteed local storage and MDM distribution.	F-018
NotificationAdapter	Transactional outbox pattern: booking and notification intent are written in one DB transaction to a pending_notifications table. A background worker polls, sends via SMS/email HTTP APIs, and marks sent. Retries on provider failure. No message queue needed.	Multiple

3.5 Architecture pattern and rationale

Chosen pattern: modular monolith with domain-driven boundaries.

Single application with clearly separated internal modules, each owning its own tables and exposing a JS module interface. Not microservices (a four-person team cannot operate independent pipelines or service mesh; the inventory invariant is easier with a single DB transaction). Not a flat layered monolith (no guidance on which service may call which; becomes tangled with nine components).

ADR-01: Pessimistic row-level locking on inventory for booking path, using SELECT … FOR UPDATE via parameterised queries through the pg driver inside a transaction. Optimistic locking at 50 concurrent req/s during Tatkal causes high retry rate. Pessimistic lock held only for one DB transaction (milliseconds).

ADR-02: All policy values in DB config tables with effective-date column. SRS requires changes without code release (RRS-F-019). Effective date lets admin set up future rules that activate automatically.

ADR-03: Transactional outbox for notification delivery. When BookingService confirms a booking, it writes the PNR row and a pending_notifications row in the same database transaction. A background worker polls this table every few seconds, sends each notification, and marks it sent. If the SMS/email provider is down, the row stays and retries next poll. This gives guaranteed-at-least-once delivery with no message queue infrastructure, just one extra table and a 20-line worker.

3.6 Technology stack

Layer	Choice	Rationale
Language +	Node.js 20 LTS, JavaScript	Team familiarity. Single language for server and front end.

Layer	Choice	Rationale
runtime		Async I/O handles the I/O-bound booking path well.
Framework	Express 4.x	Minimal, well-documented, large ecosystem. Fastify is an alternative if the team wants built-in schema validation.
Database	PostgreSQL 15	Serialisable isolation and row-level locking natively. Free on lab machines.
DB layer	pg (node-postgres) with parameterised queries	Raw driver, no ORM. The team writes SQL directly. FOR UPDATE for row-level locking. One fewer dependency to learn than Knex or Prisma.
Front end	EJS templates (server-rendered) + vanilla JS	Max 4 screens per SRS. Same language as backend, no build step needed.
Handheld	PWA (service worker + IndexedDB + html5-qrcode)	JS ecosystem, offline via service worker. IndexedDB for chart. Production: wrap in Capacitor for native APK.
Payment	Gateway hosted page + webhooks	No card data enters system.
Containerisation	Docker Compose (Node app + PostgreSQL)	Eliminates dependency mismatch across four dev machines. docker compose up gives everyone the same environment.
CI/CD	npm, GitHub Actions, SonarCloud	Required by course specification. SonarCloud has full JS support.
Testing	Jest (unit), Supertest (integration), k6 (load)	Jest is the standard JS test runner. Supertest tests Express routes. k6 generates Tatkal burst load.

3.7 Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Deadlock under concurrent booking (rows locked in different order)	Medium	High	Always lock in ascending berth-ID order. TC-BOOK-01 runs 200 concurrent requests.
Gateway callback after hold expiry: paid but no berth	Low	High	Auto refund under RRS-F-014. No silent confirmation.
Tatkal burst overwhelms connection pool	Medium	Medium	Pool sized for 50 concurrent tx. Validated by k6 (TC-PERF-01).
Chart download fails on handheld (network drop)	Medium	Medium	Retry with backoff. Shows last chart with timestamp.

3.8 Traceability to SRS requirements

Every component in Section 3.4 lists the SRS requirement IDs it implements. The RTM in SRS Section 8 maps each requirement to a design reference (DS-xxx) that points here:

Design ref (from RTM)	Component(s)	SRS requirement(s)
DS-Acc-01, DS-Acc-02	AccountService	F-001, F-002
DS-Srch-01, DS-Srch-02	SearchService, InventoryService	F-003, F-004
DS-Inv-01, DS-Inv-02	InventoryService	F-005, F-006
DS-Alloc-01, DS-Alloc-02	AllocationEngine	F-007, F-009
DS-Fare-01	FareEngine	F-010
DS-Pay-01 to DS-Pay-04	PaymentAdapter, BookingService, RefundService	F-011 to F-014
DS-Can-01, DS-Can-02	RefundService, AllocationEngine	F-015, F-016

Design ref (from RTM)	Component(s)	SRS requirement(s)
DS-Cht-01, DS-Cht-02	ChartService, HandheldApp	F-017, F-018
DS-Adm-01, DS-Aud-01	AdminConsole, AuditLog	F-019, F-020
DS-Perf-01, DS-Perf-02	BookingService, SearchService	NF-001, NF-002
DS-Rel-01	InventoryService	NF-003
DS-Sec-01 to DS-Sec-06	PaymentAdapter, AuthService, StatusAPI	NF-005, SR-001 to SR-005

3.9 Security architecture

Implements three SRS security objectives (Section 5.1.1): inventory-money agreement, data confidentiality, privileged-action accountability.

STRIDE threat model for the booking path:

Category	Threat	Control	Req
Spoofing	Attacker books using another's account	OTP-verified mobile (F-002). Second factor for clerk/admin (SR-003).	F-002, SR-003
Tampering	Fare modified between display and payment	Fare computed server-side, signed into payment request. Mismatch triggers refund.	F-010, F-014
Repudiation	Clerk denies issuing manual refund	Append-only audit log with actor, target, timestamp (F-020). Cannot be deleted.	F-020
Info disclosure	Bulk PNR harvest to track passengers	Masked details for unauthenticated lookup. 30 lookups/min rate limit.	SR-004
Denial of service	Fake booking flood during Tatkal exhausts holds	Holds auto-expire. Rate limiting. Connection pool sized for burst.	F-006, NF-001
Elevation	Passenger gains admin access	RBAC enforced server-side. Clerk cannot reach admin endpoints.	SR-003

Platform controls: TLS 1.2+ everywhere (SR-001). Hosted payment page only (SR-002). Passwords: memory-hard hash + per-account salt, lockout after 5 failures (SR-005).

4. Design

4.1 Design overview

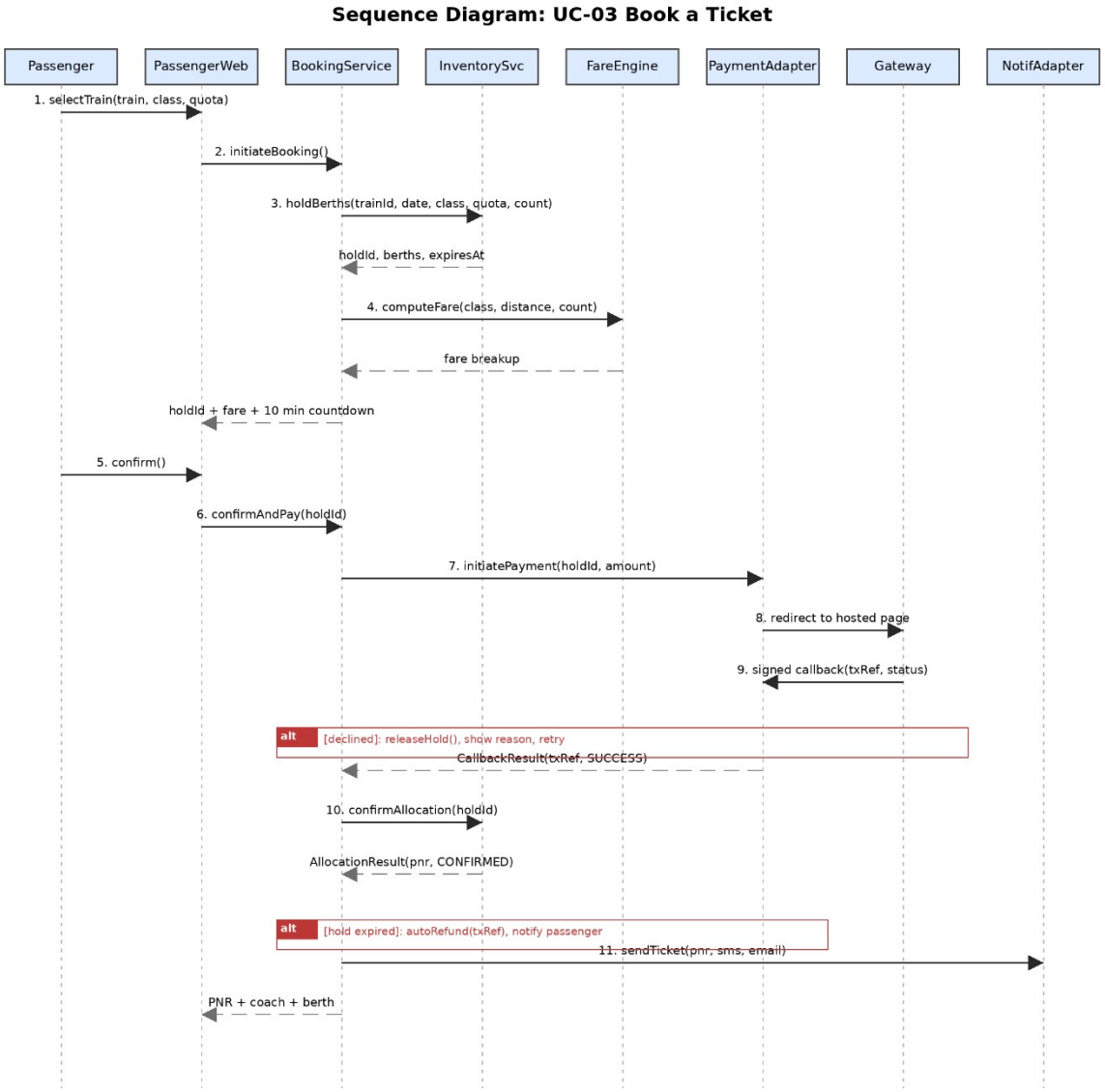
Modules map one-to-one to components in Section 3.4. Each owns its DB tables and exports a JS module interface. Cross-module calls go through these exports, never direct table access. The booking flow touches six modules in sequence:

SearchService, InventoryService, FareEngine, PaymentAdapter, InventoryService (confirm), NotificationAdapter.

Orchestration lives in BookingService. The cancellation flow touches RefundService, InventoryService, AllocationEngine, NotificationAdapter.

4.2 Sequence diagrams

Sequence Diagram 1: UC-03 Book a Ticket (main success + alt fragments)



Participants:

Passenger, *PassengerWeb*, *BookingService*, *InventoryService*, *FareEngine*, *PaymentAdapter*, *Gateway*, *NotificationAdapter*.

Flow:

(1) Passenger selects train, class, quota.

(2) PassengerWeb calls BookingService.initiateBooking().

(3) BookingService calls InventoryService.holdBerths() which acquires row-level lock and marks berths held for 10 minutes.

(4) BookingService calls FareEngine.computeFare() and gets itemised breakup.

(5) Returns hold ID, fare, countdown to passenger.

(6) Passenger confirms; BookingService calls PaymentAdapter.initiatePayment().

(7) Passenger redirected to gateway hosted page.

(8) Gateway sends signed callback to PaymentAdapter.receiveCallback().

(9) PaymentAdapter verifies signature, checks idempotency.

(10) BookingService calls InventoryService.confirmAllocation().

(11) PNR with QR generated.

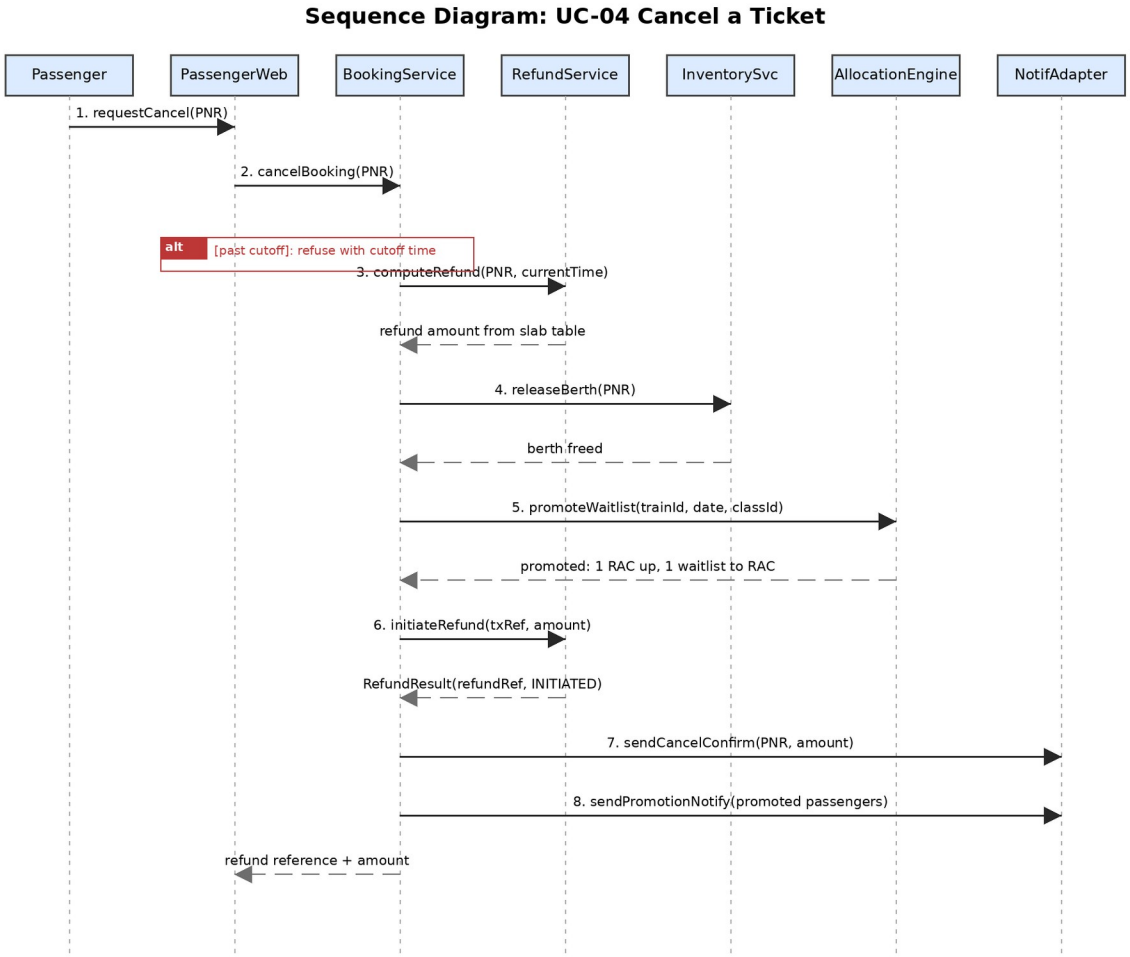
(12) NotificationAdapter sends ticket by SMS and email.

(13) PNR, coach, berth displayed.

Alt 6a (payment declined): PaymentAdapter returns failure. BookingService calls InventoryService.releaseHold(). Reason shown. Second attempt offered, availability rechecked. After two failures, booking abandoned.

Alt 10a (hold expired after payment): InventoryService.confirmAllocation returns failure. BookingService triggers auto refund (RRS-F-014). Passenger notified with refund reference. Logged with correlation ID.

Sequence Diagram 2: UC-04 Cancel a Ticket (main success)



Participants:

Passenger, *PassengerWeb*, *BookingService*, *RefundService*, *InventoryService*, *AllocationEngine*, *NotificationAdapter*.

Flow:

(1) Passenger requests cancellation.

(2) BookingService checks cutoff (RRS-F-015); refuses if past cutoff.

(3) RefundService.computeRefund() looks up slab table.

(4) InventoryService.releaseBerth() frees the berth.

(5) AllocationEngine.promoteWaitlist() moves earliest RAC up, earliest waitlisted into RAC.

(6) RefundService.initiateRefund() calls gateway.

(7) NotificationAdapter sends cancellation confirmation and promotion notifications.

(8) Refund amount and reference displayed.

4.3 API design

Two internal JS module interfaces (not REST endpoints). Express route handlers call these; they are the contract between modules.

Interface 1: InventoryService

Method	Parameters	Returns	Behaviour
holdBerths	trainId, date, classId, quotaId, count	HoldResult {holdId, berths, expiresAt}	Row-level lock. Marks berths held for 10 min. If not enough free, allocates RAC/waitlist. Throws InsufficientInventoryException.
confirmAllocation	holdId	AllocationResult {pnr, status}	Checks hold not expired. Converts to permanent. Throws HoldExpiredException.
releaseHold	holdId	void	Releases held berths. Idempotent.
releaseBerth	pnr	void	Frees berth on cancellation.

Interface 2: PaymentAdapter

Method	Parameters	Returns	Behaviour
initiatePayment	holdId, amount, currency	PaymentRedirect {redirectUrl, txRef}	Creates PENDING record. Returns gateway URL. No card data.
receiveCallback	payload, signature	CallbackResult {txRef, status}	Verifies signature. If already processed, returns stored result (idempotent). Otherwise updates record.
initiateRefund	txRef, amount	RefundResult {refundRef, status}	Calls gateway refund API. Records reference. Settlement is async.

4.4 Error handling, logging and monitoring

Every module returns a result object or throws a named Error. Express error-handling middleware catches these and maps them to HTTP responses. No stack trace returned to client. Correlation ID included in user-facing error for tracing.

Payment disagreements (money moved, allocation failed) are highest severity: handled by auto-refund (RRS-F-014), logged at ERROR with correlation ID, written to audit log.

Logging: Structured JSON via Winston or Pino. Every line: correlation ID, timestamp, module, severity. No PII in logs. PNR masked in production.

Monitoring: Health endpoint (UP/DOWN). Booking failure count per hour in metrics table. Reconciliation job (TC-REL-01) compares inventory against issued PNRs after every test run.

4.5 UX design

Passenger booking: four screens (search, selection, passenger details, payment). Fare breakup and hold countdown shown before payment (RRS-F-010, RRS-F-006). Counter: single screen, fully keyboard-operable. Handheld: coach-wise list searchable by berth/PNR, chart sync timestamp visible. All three: WCAG 2.1 AA, 4.5:1 contrast, keyboard nav. Passenger channel in English, Hindi, Kannada (RRS-NF-006).

4.6 Open issues and next steps

ID	Issue	Target
O-01	Connection pool sizing for Tatkal burst to be validated with k6	Sprint 2
O-02	Retry/backoff parameters for notification delivery not decided	Sprint 2
O-03	Confirm PWA camera access and offline chart storage on lab Android devices	Sprint 1
O-04	Mock gateway needed for idempotency integration test (TC-PAY-04)	Sprint 2

5. Appendices

5.1 Glossary

See Section 1.4 for SAD-specific terms. SRS Section 1.4 for domain terms. Additional: Express (Node.js web framework), pg/node-postgres (PostgreSQL driver), PostgreSQL (relational DB), EJS (server-side template engine), Docker Compose (multi-container dev environment), Capacitor (wraps a web app into a native mobile APK), html5-qrcode (JS library for camera-based QR scanning), transactional outbox (pattern where the app writes a notification intent into a DB table in the same transaction as the business data, and a background worker delivers it), k6 (load testing tool), SonarCloud (static analysis), Jest (JS test framework), Supertest (HTTP assertion library), PWA (Progressive Web App, works offline via a service worker).

5.2 References

IEEE 42010:2022, Systems and software engineering, Architecture description.

OWASP Top 10 (2021): https://owasp.org/www-project-top-ten/

STRIDE threat modelling: Microsoft Security Development Lifecycle.

SRS, Railway Reservation System, Team 11, v1.0, 09 September 2026.

Course mini-project handout, UE23CS341A, PES University.

5.3 Tools

draw.io for UML diagrams. Git/GitHub for version control. GitHub Projects for the backlog and sprint board. ESLint and Jest coverage in CI for static analysis and coverage. GitHub Actions for CI/CD. Swagger/OpenAPI if REST endpoints are exposed externally.

End of document. Version 1.2, Team 11.